

Practical Session 10

Task 1

Investigate inode structures using commands such as `ls -li`, `stat`, and `find`. Create hard links and symbolic links and explain their impact on inode allocation.

```
j_rakshitha@RAKSHITHA:~$ cd ~
echo "Operating Systems inode demonstration" > practical10_original.txt
j_rakshitha@RAKSHITHA:~$ ls -li practical10_original.txt
40822 -rw-r--r-- 1 j_rakshitha j_rakshitha 38 Sep 28 04:40 practical10_original.txt
j_rakshitha@RAKSHITHA:~$ stat practical10_original.txt
  File: practical10_original.txt
  size: 38          Blocks: 8          IO Block: 4096   regular file
Device: 8,48      Inode: 40822        Links: 1
Access: (0644/-rw-r--r--)  Uid: ( 1000/j_rakshitha)   Gid: ( 1000/j_rakshitha)
Access: 2026-09-28 04:40:45.606899930 +0000
Modify: 2026-09-28 04:40:45.606899930 +0000
Change: 2026-09-28 04:40:45.606899930 +0000
 Birth: 2026-09-28 04:40:45.606899930 +0000
j_rakshitha@RAKSHITHA:~$ find ~ -inum 40822
/home/j_rakshitha/practical10_original.txt
j_rakshitha@RAKSHITHA:~$ ln practical10_original.txt practical10_hardlink.txt
j_rakshitha@RAKSHITHA:~$ ls -li practical10_original.txt practical10_hardlink.txt
40822 -rw-r--r-- 2 j_rakshitha j_rakshitha 38 Sep 28 04:40 practical10_hardlink.txt
40822 -rw-r--r-- 2 j_rakshitha j_rakshitha 38 Sep 28 04:40 practical10_original.txt
j_rakshitha@RAKSHITHA:~$ ln -s practical10_original.txt practical10_symlink.txt
j_rakshitha@RAKSHITHA:~$ ls -li practical10_original.txt practical10_hardlink.txt practical
10_symlink.txt
40822 -rw-r--r-- 2 j_rakshitha j_rakshitha 38 Sep 28 04:40 practical10_hardlink.txt
40822 -rw-r--r-- 2 j_rakshitha j_rakshitha 38 Sep 28 04:40 practical10_original.txt
40837 lrwxrwxrwx 1 j_rakshitha j_rakshitha 24 Sep 28 04:42 practical10_symlink.txt -> pract
ical10_original.txt
j_rakshitha@RAKSHITHA:~$ cat practical10_original.txt
cat practical10_hardlink.txt
cat practical10_symlink.txt
Operating Systems inode demonstration
Operating Systems inode demonstration
Operating Systems inode demonstration
j_rakshitha@RAKSHITHA:~$
```

Task 2

Design a program using `mmap()` to perform file reading and writing. Compare memory-mapped I/O with traditional read/write operations in terms of efficiency and implementation complexity.

```
j_rakshitha@RAKSHITHA:~$ nano practical10_task2_mmap.c
j_rakshitha@RAKSHITHA:~$ gcc -Wall -Wextra practical10_task2_mmap.c -o practical10_task2_mm
ap
j_rakshitha@RAKSHITHA:~$ ./practical10_task2_mmap
Data written using mmap():
Memory mapped I/O demonstration
Operating Systems and System Programming

File successfully written using memory-mapped I/O.
j_rakshitha@RAKSHITHA:~$ cat practical10_mmap.txt
Memory mapped I/O demonstration
Operating Systems and System Programming
j_rakshitha@RAKSHITHA:~$ ls -l practical10_mmap.txt
-rw-r--r-- 1 j_rakshitha j_rakshitha 73 Sep 28 04:44 practical10_mmap.txt
j_rakshitha@RAKSHITHA:~$
```

```
j_rakshitha@RAKSHITHA: ~ × + ▾
GNU nano 8.7.1 practical10_task2_mmap.c
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/mman.h>
#include <sys/stat.h>
#include <string.h>

int main(void)
{
    const char *filename = "practical10_mmap.txt";
    const char *message =
        "Memory mapped I/O demonstration\n"
        "Operating Systems and System Programming\n";

    int fd = open(filename, O_RDWR | O_CREAT | O_TRUNC, 0644);

    if (fd == -1)
    {
        perror("open");
        return 1;
    }

    size_t size = strlen(message);

    if (ftruncate(fd, size) == -1)
    {
        perror("ftruncate");
        close(fd);
        return 1;
    }

    char *mapped = mmap(NULL, size,
                        PROT_READ | PROT_WRITE,
                        MAP_SHARED, fd, 0);

    if (mapped == MAP_FAILED)
    {
        perror("mmap");
        close(fd);
        return 1;
    }

    memcpy(mapped, message, size);
}

[ Read 69 lines ]
^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^_ Go To Line
```

```
if (msync(mapped, size, MS_SYNC) == -1)
{
    perror("msync");
    munmap(mapped, size);
    close(fd);
    return 1;
}

printf("Data written using mmap():\n");
printf("%.5s", (int)size, mapped);

if (munmap(mapped, size) == -1)
{
    perror("munmap");
    close(fd);
    return 1;
}

close(fd);

printf("\nFile successfully written using memory-mapped I/O.\n");

return 0;
}
```